

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	15
REFERÊNCIAS.....	16

EMSE

O QUE VEM POR AÍ?

Nesta aula vamos abordar as diferentes redes que existem no Docker, primeiramente com desenhos para melhor entendimento e depois na prática, a fim de compreender como elas realmente funcionam.

Além disso, vamos aprender a utilizar uma nova ferramenta e um novo projeto, o qual é relativo a Web Scraper com Docker Compose voltado para a bolsa de valores.



HANDS ON

Nessa aula você vai aprender como funcionam as principais redes no Docker, além de quando utilizar cada tipo de rede para determinadas situações-problema que possam vir a ocorrer no dia a dia. E de sobra vamos conhecer o Docker Swarm, um “primo” do Kubernetes.

Está esperando o quê? Temos até um projeto para portfólio. Então, te vejo nos vídeos!

EMAND

SAIBA MAIS

Docker Network e redes normais

Antes de explicar o que de fato é o Docker Network, irei utilizar uma analogia para melhor entendimento. Imagine ele como um grupo no Whatsapp ou algum outro grupo genérico de bate-papo. As pessoas (ou containers) dentro desse grupo conseguem se comunicar entre si, visto que cada uma delas tem seu respectivo nome (ou namespace), e número de celular (ou número de IP). Mas o mais importante é que cada pessoa está relacionada ou vinculada a um grupo, sendo ele a Network.

Quem está dentro desse grupo (ou Network) consegue se comunicar com quem está fora desse grupo. Entretanto, quem está fora desse grupo não sabe quais pessoas fazem parte dele e, conseqüentemente, não conseguem se comunicar com essas pessoas. Sendo assim, dependendo do grupo (ou Network) é possível impedir que pessoas externas entrem dentro daquele respectivo grupo sem um convite, tornando-o um grupo privado. Mas também é possível criar grupos abertos, em que qualquer pessoa pode entrar sem convite algum.

Fica claro, portanto, que o Docker Network nada mais é do que um recurso do Docker que tem como objetivo realizar a comunicação entre containers, funcionando como uma rede privada ou pública (depende de como você definir, vamos ver isso posteriormente) para troca de dados entre eles. Esta é uma das maneiras de realizar a comunicação de containers de forma simplificada, ajudando a manter a comunicação segura e (depende de como você definir) isolada do ambiente, o que garante que os containers realizem comunicação entre si, ou com o mundo exterior, quando você quiser. Bacana, não é mesmo?

Já uma rede "normal" é aquela que diz respeito à comunicação de uma cidade, por exemplo, que é composta por fios, roteadores e switches, entre outros equipamentos que conectam vários dispositivos como computadores, smartphones, servidores etc. Alguns exemplos seriam LAN, WAN e até mesmo a própria internet.

O Docker Network é uma rede desenvolvida para containers do Docker e funciona de forma semelhante a uma rede normal. Entretanto, ela se torna específica para containers, criando uma espécie de "mini rede privada" que permite a comunicação segura e eficiente entre eles, isolando-os de acessos externos.

Enquanto redes como LAN, WAN e internet conectam dispositivos físicos, a Docker Network conecta containers virtuais.

Agora, visando o mundo real no mercado de trabalho, imagine uma arquitetura com N serviços, aplicações, bancos e demais ferramentas em um ambiente de produção. Podemos aplicar o Docker Network em arquiteturas do tipo on-premise utilizando kubernetes, além de também poder utilizar em ambientes cloud, como GCP, AWS e Azure, entre outras clouds, isso tudo empregando VPCs.

Um pouco mais sobre tipos de redes

As redes do Docker são criadas a partir de subsistemas chamados de drivers, os quais são os responsáveis por criar a “lógica” de network no Docker. Para isso, temos alguns tipos já pré-instalados: Bridge, Host e None.

Bridge Network

Como dito anteriormente, essa é uma das redes default (padrão) do Docker que já vem pré-instalada, além de ser um driver de rede virtual. Quando criamos um container sem passar explicitamente uma rede, esse container automaticamente é vinculado à rede bridge padrão do Docker para realizar a comunicação entre os demais containers. Essa comunicação é feita por meio de uma interface virtual chamada **docker0** que tem como objetivo “simular” uma rede “normal”.

Entretanto, podemos também criar redes customizadas utilizando o driver bridge para N containers se comunicarem de forma que outros containers não possam interferir ou causar futuros conflitos. Podemos criar da seguinte forma:

```
$ docker network create --driver bridge my_network
```

Código-fonte 1 – Exemplo de comando para criar uma network do tipo bridge
Fonte: Docker (2024)

Dessa forma, criamos uma network do tipo bridge chamada de **my_network** em que podemos vincular containers novos e existentes se necessário. Um exemplo seria esse:

```
$ docker network connect my_network <CONTAINER>
```

Código-fonte 2 – Exemplo de comando para vincular um container em uma rede bridge existente
Fonte: Docker (2024)

Após executar a vinculação de um container em uma network, esse novo container é “descoberto” ou “reconhecível” para outros containers dentro dessa rede (ou grupo, voltando àquela analogia do grupo de bate-papo). Sendo assim, podemos nos comunicar utilizando namespaces (alias) ou IPs com suas respectivas portas TCP.

Para visualizar uma network, podendo ela ser de qualquer tipo, podemos utilizar o seguinte comando:

```
$ docker network inspect my_network
```

Código-fonte 3 – Exemplo de comando para visualizar uma network
Fonte: Docker (2024)

Um resultado parecido como este será exibido:

```
[
  {
    "Name": "<network_name>",
    "Id": "<network_id>",
    "Created": "<created_at>",
    "Scope": "local",
    "Driver": "bridge",
    ...
    "Containers": {
      "<id_container>": {
        "Name": "<container_name>",
        ...
      },
      ...
    }
  }
]
```

Código-fonte 4 – Exemplo de saída do comando docker inspect network
Fonte: Docker (2024)

Como podemos ver, o escopo da network bridge se envolve somente no ambiente local (scope). Sendo assim, ele não permite que demais ambientes se comuniquem por meio dessa rede (como é o caso da rede do tipo overlay, que falaremos mais a seguir). Assista à nossa aula de Docker Network para saber mais!

Host Network

A host network é uma das redes padrões do Docker, além de ser também um driver de rede. Entretanto, só é possível ter uma rede do tipo host, o que impossibilita

a criação de redes customizadas a partir desse “subsistema”, diferentemente do caso da rede bridge.

É importante reforçar que o driver ou a host network do Docker nada mais é do que um driver ou “subsistema” do Docker que conecta ou vincula demais containers com a rede física da sua máquina local. Em outras palavras, o Docker deixa de utilizar a rede virtual para utilizar a rede física já existente de onde ele está sendo executado. Essa rede pode ser de um ambiente cloud, de um servidor on-premise, de um computador ou de um simples Raspberry Pi, por exemplo.

Sendo assim, recursos do ambiente nessa infra que utiliza a rede do tipo host enxergam os containers e podem acessá-los, visto que eles utilizam as portas existentes do ambiente de onde estão sendo executados. Isso pode causar conflitos com portas TCP e demais recursos do sistema, dependendo do ambiente em que esses contêineres estão sendo executados. Assim, para conectar um container podemos utilizar o seguinte:

```
$ docker network connect my_host_network <CONTAINER>
```

Código-fonte 6 – Exemplo de comando para conectar um container
Fonte: Docker (2024)

Também podemos utilizar (para qualquer tipo de network) o seguinte comando para desconectar um container incluído em uma network existente:

```
$ docker network disconnect my_host_network <CONTAINER>
```

Código-fonte 7 – Exemplo de comando para desvincular um container em uma rede do tipo host existente
Fonte: Docker (2024)

None Network

A None Network é um driver padrão do Docker que assim como as redes bridge e host já vem pré-configurada. Como diz o nome, ela se refere à “nenhuma rede”; sendo assim, containers vinculados a essa rede não contêm IPs, alguma forma de outros containers ou até mesmo recursos do sistema, com exceção do comando **docker exec -it**.

Ela é muito útil para retenção ou proteção de dados contra desastres que possam vir a ocorrer. Podemos citar o caso da Renner, por exemplo, que teve seus respectivos sistemas e bancos de dados totalmente corrompidos. Assista às aulas para saber mais!

Para saber mais sobre os demais tipos de redes

Como não abordamos nas aulas em vídeo, iremos agora falar sobre networks como macvlan e ipvlan.

Macvlan Network

É um tipo de rede no Docker que utiliza endereços MAC únicos para cada container, sendo esses endereços IPs utilizados no mesmo espaço do host (ou máquina local, ambiente). Isso faz com que cada container pareça um dispositivo físico separado, facilitando a comunicação direta com demais dispositivos na mesma rede. É útil em cenários em que se deseja isolar tráfego ou vincular containers com uma infraestrutura de rede existente.

O Grupo Globo ainda possui alguns servidores do tipo on-premise. Antes da migração em massa para o ambiente cloud (GCP), o grupo dependia muito do bom gerenciamento de IPs para distribuir para os demais times criarem novas aplicações, por exemplo. Isso não significa que se utilizava especificamente o tipo Macvlan, mas é um caso de situação problema que pode vir a ocorrer devido à falta de IPs disponíveis. Podemos criar uma rede desse tipo da seguinte maneira:

```
$ docker network create -d macvlan \
  --subnet=192.168.1.0/24 \
  --gateway=192.168.1.1 \
  -o parent=eth0 \
  my_macvlan_network
```

Código-fonte 8 – Exemplo de comando para criar uma rede do tipo macvlan
Fonte: Docker (2024)

Ipvlan Network

A Ipvlan Network é parecida com a Macvlan, mas uma diferença é que todos os containers compartilham o mesmo endereço MAC do host. Entretanto, cada container pode ter seu respectivo endereço IP, simplificando a comunicação e o

tráfego para redes externas, suportando modos principais, como L2 e L3, e controlando assim o roteamento de tráfego.

```
$ docker network create -d ipvlan \  
  --subnet=10.0.0.0/24 \  
  --gateway=10.0.0.1 \  
  -o parent=eth0 \  
  my_ipvlan_network
```

Código-fonte 9 – Exemplo de comando para criar uma rede do tipo ipvlan
Fonte: Docker (2024)

Overlay Network e Docker Swarm

Só é possível utilizar o tipo de rede Overlay em ambientes para aplicações containerizadas, como por exemplo, empregando orquestradores como Docker Swarm ou Kubernetes. Sendo assim, sem um orquestrador, como o Swarm, não há necessidade de utilizar uma rede Overlay, uma vez que esse tipo de rede depende de uma coordenação e somente orquestradores oferecem suporte nativo para ele.

O objetivo dessa rede é criar redes distribuídas permitindo que diferentes hosts realizem a comunicação como se fossem um único host. Além disso, a carga de trabalho é dividida entre os N hosts conectados nessa rede overlay, possibilitando assim uma escalabilidade horizontal facilmente.

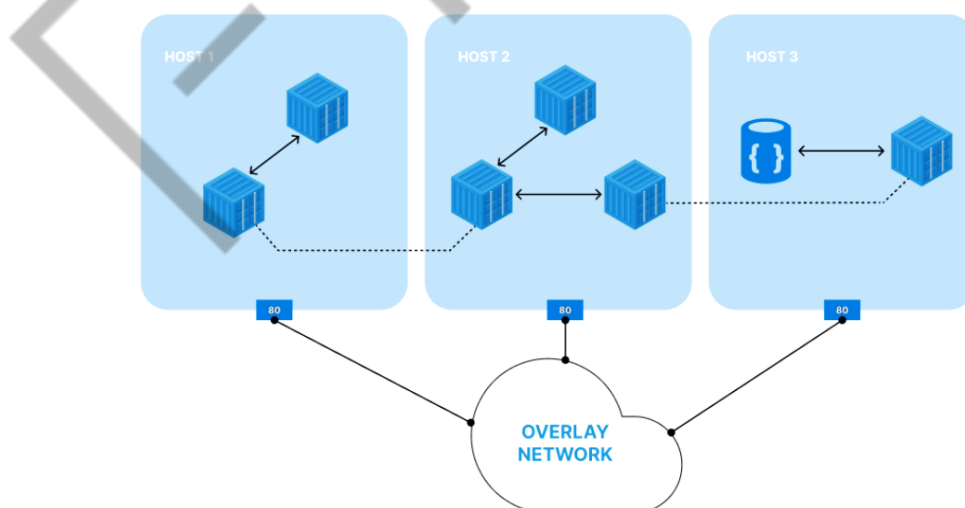


Figura 1 - Demonstração de uma rede do tipo overlay com hosts diferentes conectados
Fonte: Elaborado pelo autor (2024)

Ambientes desse tipo atualmente são frequentemente utilizados, nos quais podemos ter uma máquina em execução em São Paulo, outra em Carolina do Sul e outra em Vancouver, entre outros lugares do mundo. Isso possibilita uma rápida entrega para o usuário final com uma baixa latência, pois imagine um usuário que está em Portugal e tenta ler as notícias do site O Globo ou do G1: a experiência dele não seria das melhores.



Figura 2 - Demonstração de regiões utilizando o tipo de rede overlay com diferentes hosts conectados
Fonte: Google Cloud (2024)

Além disso, com essa disseminação de hosts temos uma melhor distribuição de carga; se por algum motivo um ponto cair, temos os demais para servir o produto, oferecendo maior segurança e estabilidade do sistema.

Para iniciar uma rede do tipo overlay em sua máquina local, primeiramente deve-se executar o comando de inicialização do Docker Swarm que já vem na instalação do Docker.

```
$ docker swarm init
```

Código-fonte 10 – Exemplo de comando para inicializar o orquestrador Docker Swarm
Fonte: <https://docs.docker.com/network/>

Em seguida, após a inicialização do Docker Swarm. Podemos executar o comando a seguir para visualizar as conexões existentes:

```
$ docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
409c320ea617	bridge	bridge	local
1379cced7f19	host	host	local
1bdda1a8a3c7	none	null	local

Código-fonte 11 – Exemplo de comando para listar as redes do Docker existentes
Fonte: Elaborado pelo autor (2024)

Na saída, veremos que temos duas redes que o Docker Swarm acabou de criar, sendo elas uma do tipo Overlay e outra do tipo Bridge:

```
$ docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
409c320ea617	bridge	bridge	local
c444ab2bfd4c	docker_gwbridge	bridge	local
1379cced7f19	host	host	local
mba2coldec6n	ingress	overlay	swarm
1bdda1a8a3c7	none	null	local

Código-fonte 12 – Exemplo de duas redes do Docker Swarm
Fonte: Elaborado pelo autor (2024)

O Docker Swarm depende de uma rede do tipo Bridge customizada para comunicação local dentro de um node (ou nó), permitindo a flexibilidade com a comunicação interna de um cluster Docker Swarm. Ele é útil para isolamento de serviços, além de manter uma organização clara da rede. Se ficou curioso(a), recomendo fortemente reassistir nossas aulas de Docker Network.

Um pouco mais sobre redes no Docker Compose

Já conhecemos o Docker Compose e sabemos suas funcionalidades e como utilizar, além de empregar ferramentas como o Kompose para conversão de arquivos do Docker Compose para manifestos de orquestradores, como por exemplo o Kubernetes.

Sendo assim, também é possível declarar tipos de redes em arquivos do Docker Compose, facilitando nosso dia a dia e configuração. No exemplo da nossa última aula, utilizamos alguns serviços para demonstrar como podemos lançar mão de diferentes e mais de um tipo de rede em N serviços.

Também vimos que para definir redes do tipo Bridge, podemos declarar em nosso docker-compose.yml da seguinte forma:

```
...
networks:
  bridge_network:
    name: bridge_network
    driver: bridge
...
```

Código-fonte 13 – Exemplo de como declarar uma rede do tipo bridge no docker-compose.yml
Fonte: Elaborado pelo autor (2024)

E para utilizar em serviços podemos declarar da seguinte forma:

```
version: '3.8'

services:
  mongodb:
    ...
    networks:
      - bridge_network
    ...
```

Código-fonte 14 – Exemplo de como usar uma rede do tipo bridge declarada no docker-compose.yml
Fonte: Elaborado pelo autor (2024)

Assim como também vimos que, para definir redes do tipo Overlay, podemos declarar da seguinte forma em nosso docker-compose.yml:

```
...
networks:
  bridge_network:
    ...
  overlay_network:
    name: overlay_network
    driver: overlay
    attachable: true
    ...
```

Código-fonte 15 – Exemplo de como declarar uma rede do tipo overlay no docker-compose.yml
Fonte: Elaborado pelo autor (2024)

Em que podemos utilizar ambas as networks em um único serviço da seguinte maneira:

```
version: '3.8'

services:
  mongodb:
    ...
    networks:
      - bridge_network
      - overlay_network
    ...
```

Código-fonte 14 – Exemplo de como utilizar ambas as networks em um único serviço Fonte:
Elaborado pelo autor (2024)

Vimos também que para definir uma rede do tipo none podemos fazer da seguinte forma, sem a necessidade de usar o “atributo” network e vinculação de portas, visto que o container não se comunica com nenhum outro serviço:

```
version: '3.8'

services:
  ...
  data-handler:
    ...
    network_mode: none
    ...
  ...
```

Código-fonte 14 – Exemplo de como definir uma rede do tipo none
Fonte: Elaborado pelo autor (2024)

O QUE VOCÊ VIU NESTA AULA?

Nessa aula você aprendeu o que é o Docker Network e como utilizá-lo em diferentes tipos de redes disponibilizados pelo Docker. Além disso, conhecemos um pouco mais sobre o Docker Swarm e simulamos três hosts virtuais de forma muito simples, além de realizar vários testes de comunicação e passagens por situações problemas que geralmente acontecem no dia a dia.

E para melhorar mais ainda, por meio do Dockerfile criamos algumas imagens customizadas voltadas para funcionalidades de dump e restore do MongoDB. Convertemos uma ferramenta, chamada mongodb-tools, para uma imagem Docker, deixando o nosso sistema limpo e sem a necessidade de instalação de ferramentas, além de evitar incompatibilidades de sistemas.

Sem falar no projeto de portfólio em que teremos uma base de dados de ações do Ibovespa que se atualiza automaticamente e nos permite analisar os dados da maneira que quisermos utilizando o streamlit.

Parabéns por ter chegado até aqui! Agora você tem três projetos de portfólio, sendo um de Machine Learning, outro de Deep Learning e outro de Análise de Dados, sem falar no ótimo conteúdo sobre containers. Se esforce para tornar-se um(a) profissional cada vez mais completo(a), não desista. E nos vemos por aí!

Ah, lembre-se que estamos disponíveis na nossa comunidade do Discord para tirar dúvidas e ajudar; além disso, o conteúdo estará sempre disponível quando você sentir a necessidade de revisitá-lo!

REFERÊNCIAS

DOCKER DOCS. **Get Docker**. 2024. Disponível em: <<https://docs.docker.com>>. Acesso em: 28 ago. 2024.

DOCKER DOCS. **Networking overview**. 2024. Disponível em: <<https://docs.docker.com/engine/network/>>. Acesso em: 29 ago. 2024.

DOCKER DOCS. **Swarm mode overview**. 2024. Disponível em: <<https://docs.docker.com/engine/swarm/>>. Acesso em: 29 ago. 2024.

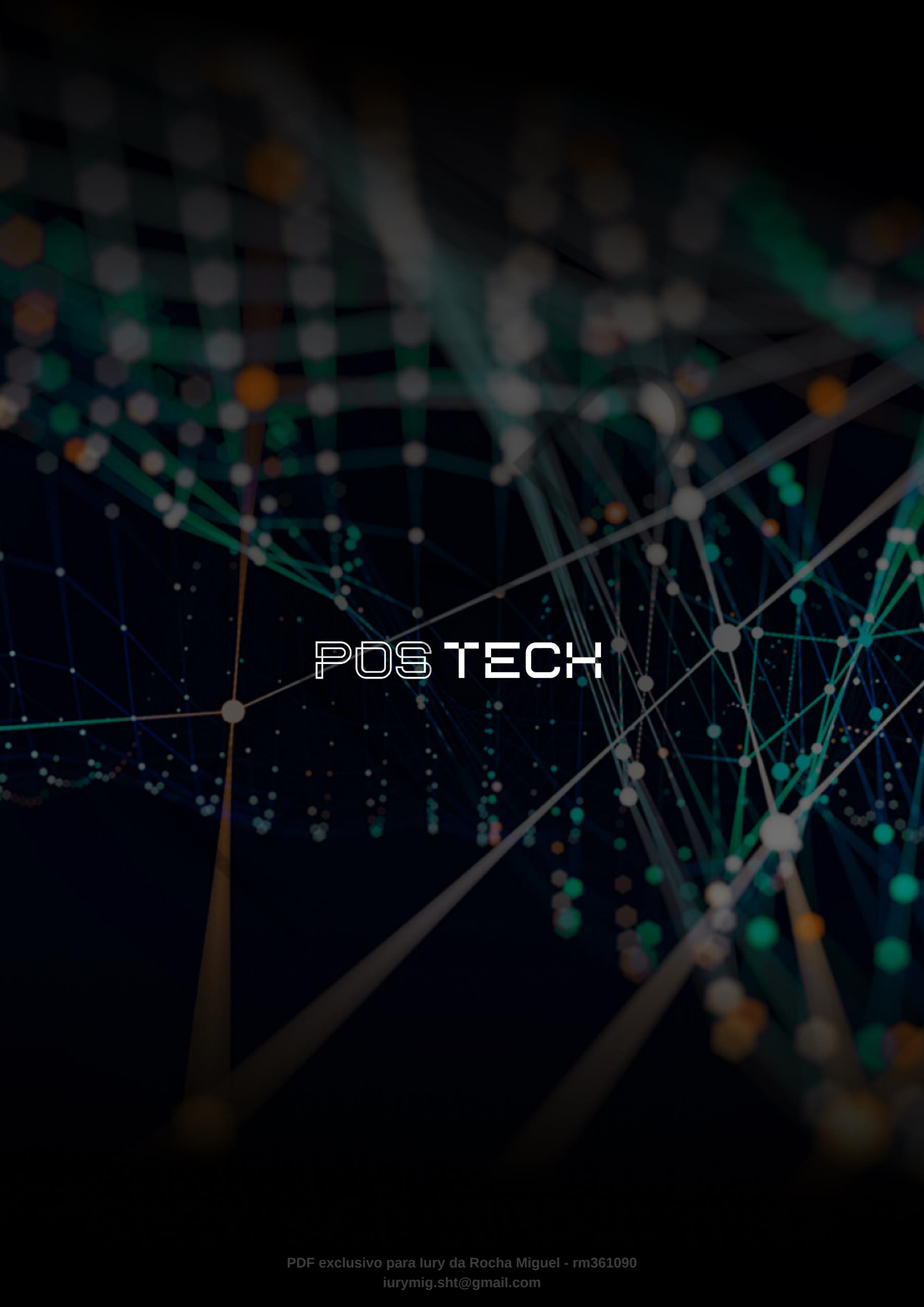
GOOGLE CLOUD. **Documentação do Google Cloud**. 2024. Disponível em: <<https://cloud.google.com/docs?hl=pt-br>>. Acesso em: 29 ago. 2024.

PAULO, W. **Docker e Docker Compose**. 2020. Disponível em: <<https://medium.com/@willianpaulo99/docker-e-docker-compose-bf967ffa84b3>>. Acesso em: 29 ago. 2024.

PALAVRAS-CHAVE

Palavras-chave: Kubernetes. Docker. Containers. Docker Compose. Docker Swarm. Overlay. Bridge. Docker Network.

EMSE



POSTECH